

/*

-----French-----

Exercice4 :

Selon Collatz, si nous continuons à itérer N comme suit :

$N=N/2$ Si N est pair

$N=3*N+1$ Si N est impair

Le nombre terminera par converger vers 1 quel que soit N.

Exemple :

N = 13

La séquence des Nombres Hailstone de 13 est : 40 20 10 5 16 8 4 2 1

Nombre des étapes requises : 9

1). Ecrire une fonction récursive *Hailstone_numbers* qui permet de générer tous les nombres Hailstone à partir de N, et de trouver le nombre d'étapes effectuées par N pour le réduire à 1

2). Allouer dynamiquement la mémoire pour

-----English-----

Exercise4:

According to Collatz, if we continue to iterate N as follows:

$N=N/2$ Si N est pair

$N=3*N+1$ Si N est impair

The number will eventually converge to 1 regardless of N.

Example :

N=13

The Hailstone number sequence of 13 is: 40 20 10 5 16 8 4 2 1

Number of steps required: 9

1). Write a recursive function *Hailstone_numbers* which generates all Hailstone numbers from N, and finds the number of steps taken by N to reduce it to 1

2). Dynamically allocate memory for

*/

```
#include <stdio.h>

#include <stdlib.h>

int hailstone_x(int x);

void hailstone_sequence(int x);

int main()
{
    int x;
    printf("\nEnter number: ");
    scanf("%d", &x);
    hailstone_sequence(x);
    return 0;
}

int hailstone_x(int x)
{
    if (x % 2 == 0)
        x = x / 2;
    else
        x = (3 * x) + 1;
    return x;
}

void hailstone_sequence(int x)
{
    int *array, length = 1, i;
    array = (int *)malloc(length * sizeof(int));
    i = 0;
    printf("\nSequence is:_____ \n");
    while (hailstone_x(x) != 1)
    {
        length++;
        array[i] = hailstone_x(x);
        printf("%d\t", *(array + i));
        x = hailstone_x(x);
        i++;
    }
    printf("%d\n", 1);
    printf("It takes us %d steps.\n\n", length);
}
```

/*

Edit by: MOHAMMED ELIDRISSI LAOUKILI
#idriCoding | #idriLogic | #idri
2021/2022

لا اله الا الله

*/